9.2 Implementing a Priority Queue

In this section, we show how to implement a priority queue by storing its entries in a positional list L. (See Section 7.4.) We provide two realizations, depending on whether or not we keep the entries in L sorted by key.

9.2.1 The Composition Design Pattern

One challenge in implementing a priority queue is that we must keep track of both an element and its key, even as items are relocated within our data structure. This is reminiscent of a case study from Section 7.6 in which we maintain access counts with each element. In that setting, we introduced the ***composition design pattern***, defining an _Item class that assured that each element remained paired with its associated count in our primary data structure.

For priority queues, we will use composition to store items internally as pairs consisting of a key k and a value v. To implement this concept for all priority queue implementations, we provide a PriorityQueueBase class (see Code Fragment 9.1) that includes a definition for a nested class named _Item. We define the syntax a < b, for item instances a and b, to be based upon the keys.

```
1  class PriorityQueueBase:
2    """Abstract base class for a priority queue."""
3
4    class _Item:
5      """Lightweight composite to store priority queue items."""
6      __slots__ = '_key', '_value'
7
8      def __init__(self, k, v):
9        self._key = k
10       self._value = v
11
12     def __lt__(self, other):
13       return self._key < other._key       # compare items based on their keys
14
15   def is_empty(self):                     # concrete method assuming abstract len
16     """Return True if the priority queue is empty."""
17     return len(self) == 0
```

Code Fragment 9.1: A PriorityQueueBase class with a nested _Item class that composes a key and a value into a single object. For convenience, we provide a concrete implementation of is_empty that is based on a presumed __len__ impelementation.

9.2.2 Implementation with an Unsorted List

In our first concrete implementation of a priority queue, we store entries within an *unsorted list*. Our UnsortedPriorityQueue class is given in Code Fragment 9.2, inheriting from the PriorityQueueBase class introduced in Code Fragment 9.1. For internal storage, key-value pairs are represented as composites, using instances of the inherited _Item class. These items are stored within a PositionalList, identified as the _data member of our class. We assume that the positional list is implemented with a doubly-linked list, as in Section 7.4, so that all operations of that ADT execute in $O(1)$ time.

We begin with an empty list when a new priority queue is constructed. At all times, the size of the list equals the number of key-value pairs currently stored in the priority queue. For this reason, our priority queue __len__ method simply returns the length of the internal _data list. By the design of our PriorityQueueBase class, we inherit a concrete implementation of the is_empty method that relies on a call to our __len__ method.

Each time a key-value pair is added to the priority queue, via the add method, we create a new _Item composite for the given key and value, and add that item to the end of the list. Such an implementation takes $O(1)$ time.

The remaining challenge is that when min or remove_min is called, we must locate the item with minimum key. Because the items are not sorted, we must inspect all entries to find one with a minimum key. For convenience, we define a nonpublic _find_min utility that returns the *position* of an item with minimum key. Knowledge of the position allows the remove_min method to invoke the delete method on the positional list. The min method simply uses the position to retrieve the item when preparing a key-value tuple to return. Due to the loop for finding the minimum key, both min and remove_min methods run in $O(n)$ time, where n is the number of entries in the priority queue.

A summary of the running times for the UnsortedPriorityQueue class is given in Table 9.1.

Operation	Running Time
len	$O(1)$
is_empty	$O(1)$
add	$O(1)$
min	$O(n)$
remove_min	$O(n)$

Table 9.1: Worst-case running times of the methods of a priority queue of size n, realized by means of an unsorted, doubly linked list. The space requirement is $O(n)$.

```
1  class UnsortedPriorityQueue(PriorityQueueBase): # base class defines _Item
2    """A min-oriented priority queue implemented with an unsorted list."""
3
4    def _find_min(self):                    # nonpublic utility
5      """Return Position of item with minimum key."""
6      if self.is_empty( ):                  # is_empty inherited from base class
7        raise Empty('Priority queue is empty')
8      small = self._data.first( )
9      walk = self._data.after(small)
10     while walk is not None:
11       if walk.element( ) < small.element( ):
12         small = walk
13       walk = self._data.after(walk)
14     return small
15
16   def __init__(self):
17     """Create a new empty Priority Queue."""
18     self._data = PositionalList( )
19
20   def __len__(self):
21     """Return the number of items in the priority queue."""
22     return len(self._data)
23
24   def add(self, key, value):
25     """Add a key-value pair."""
26     self._data.add_last(self._Item(key, value))
27
28   def min(self):
29     """Return but do not remove (k,v) tuple with minimum key."""
30     p = self._find_min( )
31     item = p.element( )
32     return (item._key, item._value)
33
34   def remove_min(self):
35     """Remove and return (k,v) tuple with minimum key."""
36     p = self._find_min( )
37     item = self._data.delete(p)
38     return (item._key, item._value)
```

Code Fragment 9.2: An implementation of a priority queue using an unsorted list. The parent class PriorityQueueBase is given in Code Fragment 9.1, and the PositionalList class is from Section 7.4.

9.2.3 Implementation with a Sorted List

An alternative implementation of a priority queue uses a positional list, yet maintaining entries sorted by nondecreasing keys. This ensures that the first element of the list is an entry with the smallest key.

Our SortedPriorityQueue class is given in Code Fragment 9.3. The implementation of min and remove_min are rather straightforward given knowledge that the first element of a list has a minimum key. We rely on the first method of the positional list to find the position of the first item, and the delete method to remove the entry from the list. Assuming that the list is implemented with a doubly linked list, operations min and remove_min take $O(1)$ time.

This benefit comes at a cost, however, for method add now requires that we scan the list to find the appropriate position to insert the new item. Our implementation starts at the end of the list, walking backward until the new key is smaller than an existing item; in the worst case, it progresses until reaching the front of the list. Therefore, the add method takes $O(n)$ worst-case time, where n is the number of entries in the priority queue at the time the method is executed. In summary, when using a sorted list to implement a priority queue, insertion runs in linear time, whereas finding and removing the minimum can be done in constant time.

Comparing the Two List-Based Implementations

Table 9.2 compares the running times of the methods of a priority queue realized by means of a sorted and unsorted list, respectively. We see an interesting trade-off when we use a list to implement the priority queue ADT. An unsorted list supports fast insertions but slow queries and deletions, whereas a sorted list allows fast queries and deletions, but slow insertions.

Operation	Unsorted List	Sorted List
len	$O(1)$	$O(1)$
is_empty	$O(1)$	$O(1)$
add	$O(1)$	$O(n)$
min	$O(n)$	$O(1)$
remove_min	$O(n)$	$O(1)$

Table 9.2: Worst-case running times of the methods of a priority queue of size n, realized by means of an unsorted or sorted list, respectively. We assume that the list is implemented by a doubly linked list. The space requirement is $O(n)$.

```
1  class SortedPriorityQueue(PriorityQueueBase): # base class defines _Item
2    """A min-oriented priority queue implemented with a sorted list."""
3
4    def __init__(self):
5      """Create a new empty Priority Queue."""
6      self._data = PositionalList( )
7
8    def __len__(self):
9      """Return the number of items in the priority queue."""
10     return len(self._data)
11
12   def add(self, key, value):
13     """Add a key-value pair."""
14     newest = self._Item(key, value)              # make new item instance
15     walk = self._data.last( )          # walk backward looking for smaller key
16     while walk is not None and newest < walk.element( ):
17       walk = self._data.before(walk)
18     if walk is None:
19       self._data.add_first(newest)               # new key is smallest
20     else:
21       self._data.add_after(walk, newest)         # newest goes after walk
22
23   def min(self):
24     """Return but do not remove (k,v) tuple with minimum key."""
25     if self.is_empty( ):
26       raise Empty('Priority queue is empty.')
27     p = self._data.first( )
28     item = p.element( )
29     return (item._key, item._value)
30
31   def remove_min(self):
32     """Remove and return (k,v) tuple with minimum key."""
33     if self.is_empty( ):
34       raise Empty('Priority queue is empty.')
35     item = self._data.delete(self._data.first( ))
36     return (item._key, item._value)
```

Code Fragment 9.3: An implementation of a priority queue using a sorted list. The parent class PriorityQueueBase is given in Code Fragment 9.1, and the PositionalList class is from Section 7.4.